

Specification Driven Development

Développer à l'ère des LLMs

Philippe Bousquet – Architecte & Leader Java Community France
Benjamin Barbe – Expert Technique Java & DevOps

Qui sommes-nous ?



Philippe Bousquet

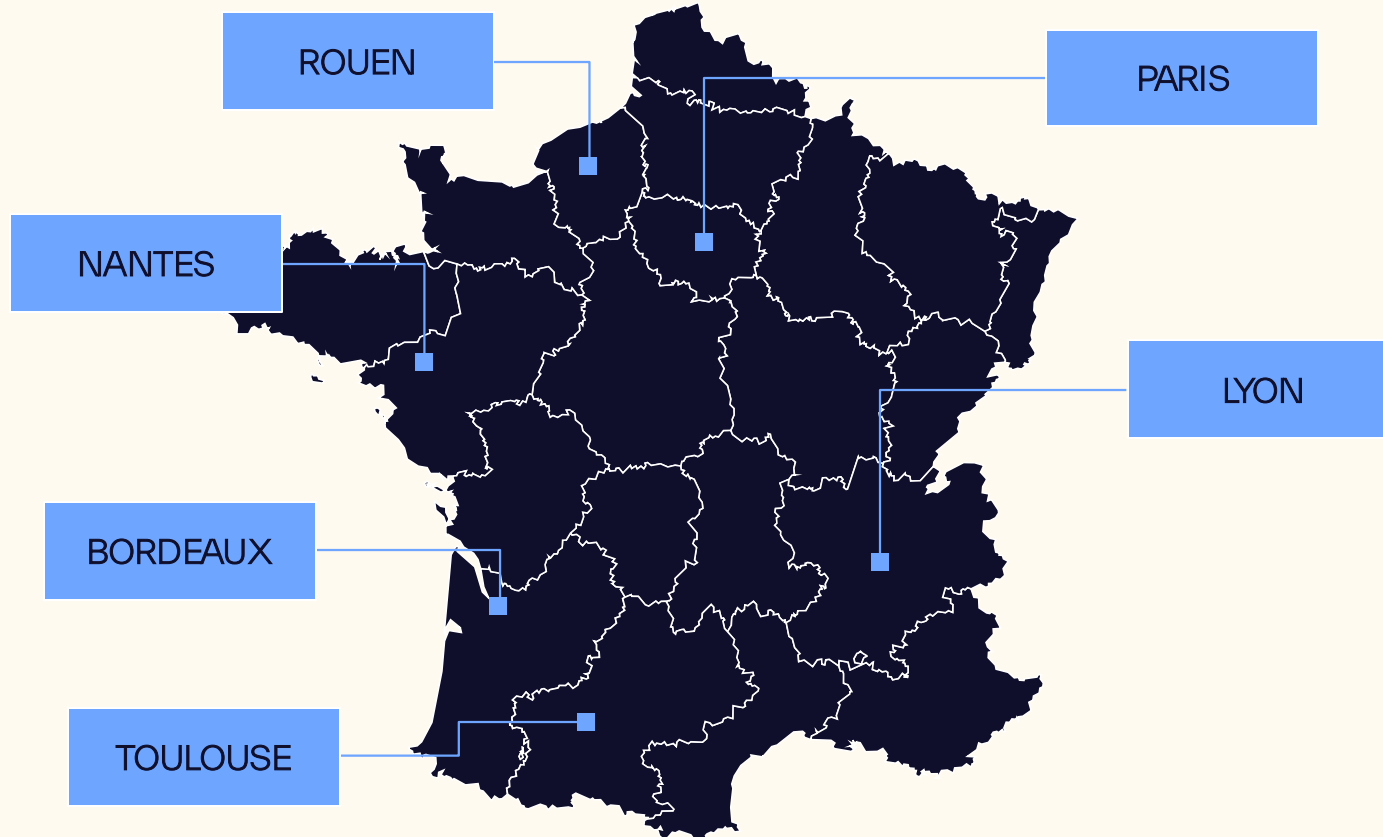
- Architecte & Leader Java Community France
- Passionné de dev depuis mes 9 ans
- Près de 30 d'expérience professionnelle
- Pilote la Veille & l'Innovation Java



Benjamin Barbe

- Expert Technique Java & DevOps
- 10 ans d'expérience
- Bricoleur sur mon homelab à mes heures perdues
- Participe à la Veille & l'Innovation Java

SQLI : <https://www.sqli.com/fr-fr>



1990

Création

2500

Employés

6

Agences en France



Specification Driven Development

Un peu de théorie

Specification-Driven Development

Concevez des logiciels à partir d'intentions explicites, et non à partir de suppositions.

Définition :

Le développement piloté par les spécifications est une approche dans laquelle des **spécifications claires et structurées** deviennent la **principale source de vérité**, et le **code est généré** en conséquence, et non comme point de départ.

Changer de rôle

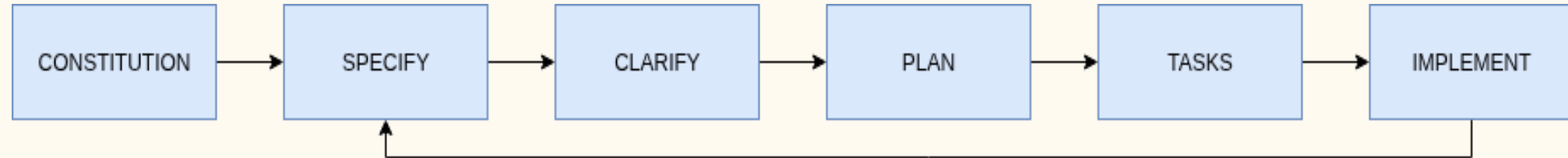
De développeur assisté... à ingénieur augmenté

Github Copilot :

- Compréhension du métier et de l'architecture
- Génération structurée (tests, docs, schémas)
- Détection proactive des incohérences
- Support clé de la phase de clarification

Spec-Kit

Un framework qui discipline l'IA



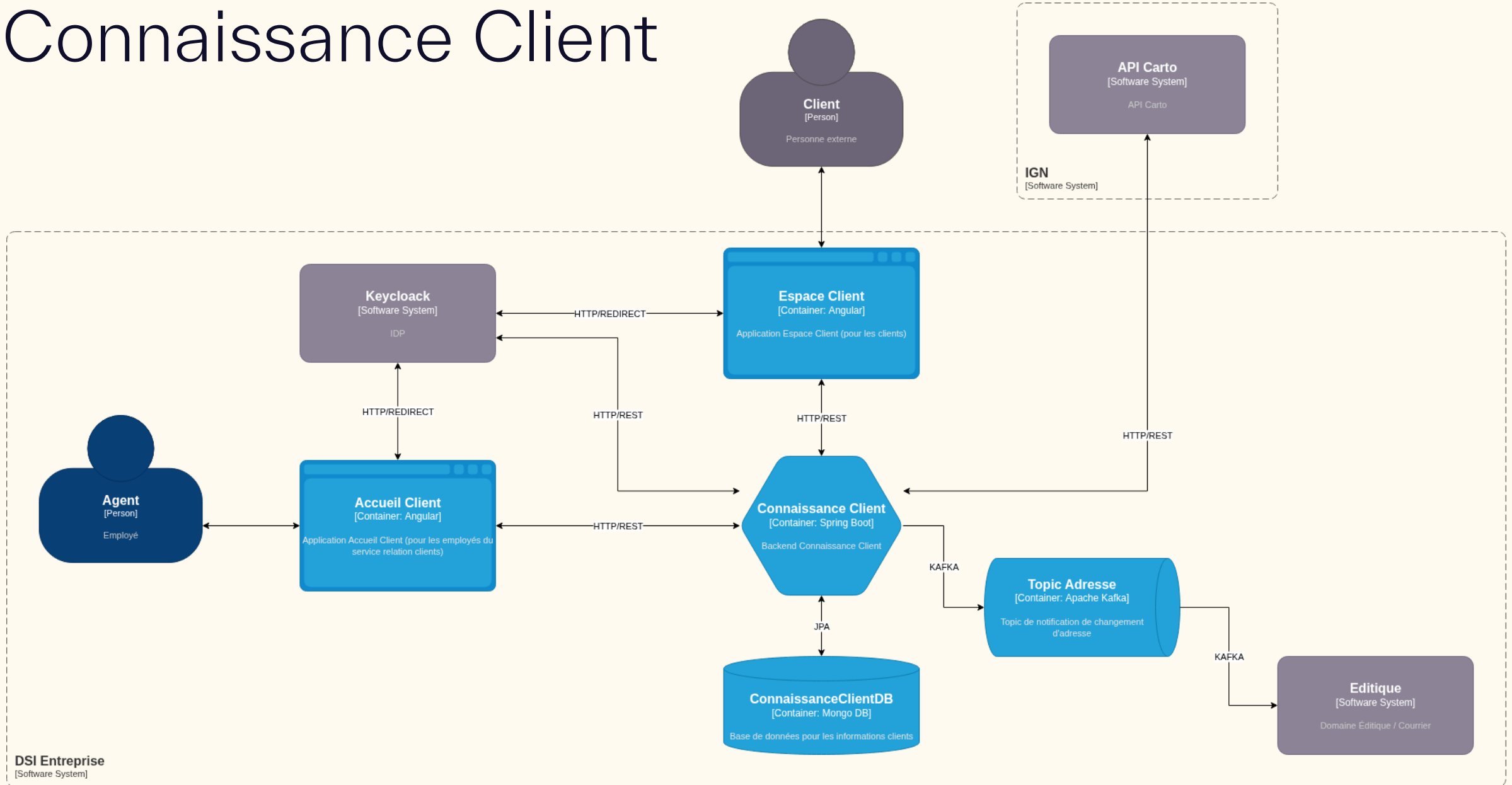
- Constitution : règles & architecture
- Spécification : intention fonctionnelle
- Clarification : éliminer les ambiguïtés
- Plan & tâches : livrer de façon prédictible
- Implémentation : génération itérative



Notre champ d'expérimentation

Connaissance Client

Connaissance Client



Les prérequis

Ce dont vous aurez besoin

- Git
- Un IDE avec Github Copilot (ou un autre assistant IA)
- Un OS supportant Docker (Linux recommandé)
- Docker
- Java 21, Maven
- React ou Angular, NPM
- Des connaissances en développement (préférable)

Récupérer le Backend

<https://github.com/darken33/connaissance-client>



Setup environnement de test

Dans le répertoire `./env-tests/docker` exécutez la commande `./start.sh`

```
pbousquet@B0-8GFFVL3:~/Workspaces/SQLI/POC/connaissance-client/env-tests/docker$ ./start.sh
[+] Running 6/6
✓ Network docker_default          Created
✓ Container local-mongodb         Started
✓ Container local-zookeeper       Started
✓ Container local-broker          Started
✓ Container local-schema-registry Started
✓ Container local-control-center  Started
COSMOSDB_DATABASE_NAME=admin
COSMOSDB_CONNECTION_STRING=mongodb://mongoadmin:secret@0.0.0.0:27017
KAFKA_BOOTSTRAP_SERVER=broker:9092
KAFKA_CONTROL_CENTER_URL=http://0.0.0.0:9021
KAFKA_SCHEMA_REGISTRY_URL=http://0.0.0.0:8090
```

Build de l'application

Dans le répertoire racine `./` exécutez la commande `./mvnw clean package`

```
[INFO] -----
[INFO] Reactor Summary for connaissance-client 2.2.0-SNAPSHOT:
[INFO]
[INFO] connaissance-client ..... SUCCESS [ 0.712 s]
[INFO] connaissance-client-domain ..... SUCCESS [ 6.453 s]
[INFO] connaissance-client-db-adapter ..... SUCCESS [ 4.664 s]
[INFO] connaissance-client-cp-adapter ..... SUCCESS [ 10.905 s]
[INFO] connaissance-client-event-adapter ..... SUCCESS [ 1.895 s]
[INFO] connaissance-client-api ..... SUCCESS [ 13.462 s]
[INFO] connaissance-client-app ..... SUCCESS [ 35.349 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:13 min
[INFO] Finished at: 2026-02-22T18:26:48+01:00
[INFO] -----
pbousquet@B0-8GFFVL3:~/Workspaces/SQLI/POC/connaissance-client$
```

Exécuter l'application

<http://localhost:8080/swagger-ui/index.html>

```
java -jar ./connaissance-client-app/target/connaissance-client-app-2.2.0-SNAPSHOT.jar
```

The screenshot shows the Swagger UI interface for the 'Connaissance Client' API. At the top, there's a Swagger logo and a search bar containing '/v3/api-docs'. Below this, the API title 'Connaissance Client' is displayed with version 'v1' and 'OAS 3.1'. A 'Servers' section shows a dropdown menu with 'http://localhost:8080 - Generated server url' and an 'Authorize' button. The main section lists several API endpoints with their methods and descriptions:

- PUT** `/v1/connaissance-clients/{id}/situation`: Modification complète de la situation familiale (recommandé)
- PUT** `/v1/connaissance-clients/{id}/adresse`: Modification complète de l'adresse du client (recommandé)
- GET** `/v1/connaissance-clients`: Consultation de toutes les fiches de connaissance client
- POST** `/v1/connaissance-clients`: Création d'une nouvelle fiche de connaissance client
- GET** `/v1/connaissance-clients/{id}`: Consultation d'une fiche de connaissance client spécifique
- DELETE** `/v1/connaissance-clients/{id}`: Suppression définitive d'une fiche de connaissance client

Below the endpoints, there's a 'Schemas' section showing the 'ConnaissanceClientDto' schema with an 'Expand all' button and the type 'object'.



Accueil Client

Créer une nouvelle application (Frontend)

Initialisation de Spec-Kit

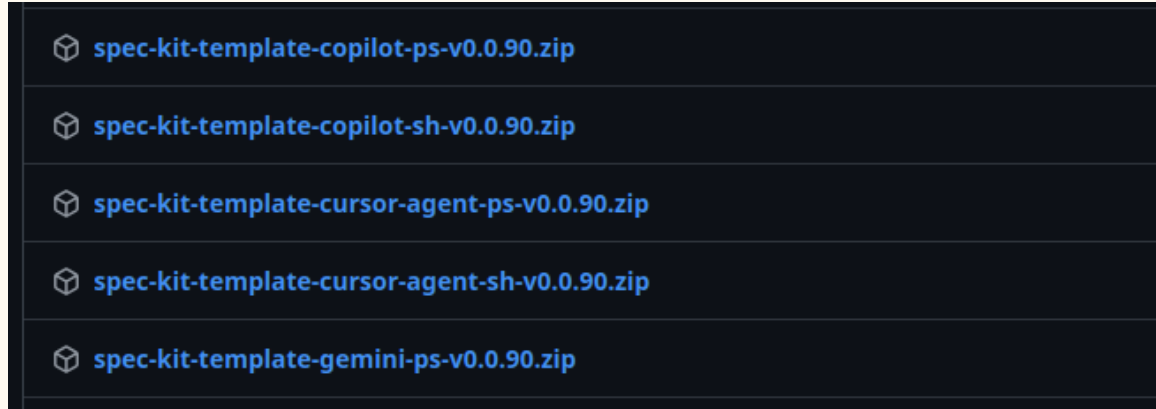
Via la CLI

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

```
specify init accueil-client
```

Ou installation Manuelle

Depuis le repo : <https://github.com/github/spec-kit/tags>

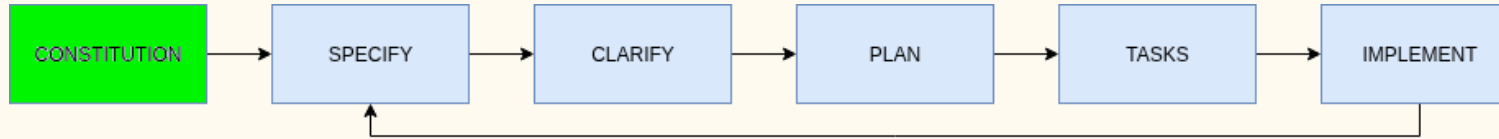


Décompresser l'archive dans votre projet.



```
✓ ACCUEIL-CLIENT
  ✓ .github
    ✓ agents
      ≡ speckit.analyze.agent.md
      ≡ speckit.checklist.agent.md
      ≡ speckit.clarify.agent.md
      ≡ speckit.constitution.agent.md
      ≡ speckit.implement.agent.md
      ≡ speckit.plan.agent.md
      ≡ speckit.specify.agent.md
      ≡ speckit.tasks.agent.md
      ≡ speckit.taskstoissues.agent.md
    > prompts
  ✓ .specify
    > scripts
    > templates
    > .vscode
```


Constitution



Exemple de prompt :

```
/speckit.constitution En tant qu'architecte technique spécialisé en Angular et TypeScript, définis la structure, les principes architecturaux et de développement d'une application front-end moderne. Pour ce faire, suis les meilleures pratiques actuelles, en respectant au minimum les principes suivants :
```

- * Utiliser les dernières versions d'Angular et de TypeScript.
- * Principe KISS : Privilégier la simplicité.
- * Séparer explicitement l'interface utilisateur du traitement.

S'agissant d'un POC, n'ayant pas pour but de partir en production, on ne traitera pas les éléments suivants :

- * Authentification et Sécurité
- * Pipelines de CI/CD

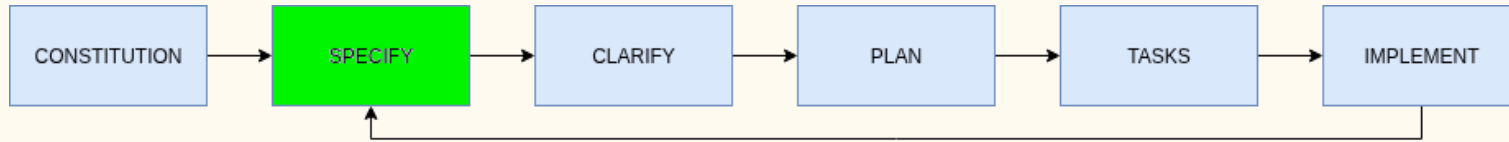
Définis uniquement le fichier constitution.md, puis me le retourner afin que je puisse vérifier le respect de ces principes.



Principes attendus :

- Angular & TypeScript à jour
- KISS (Keep It Simple, Stupid)
- Séparation UI vs Traitement
- Mode POC

Spécifications



Exemple de prompt :

```
/speckit.specify Nous disposons actuellement d'une application backend exposant des API REST définies dans le contrat OpenAPI (fichier `connaissance-client-api.yaml`). En tant qu'architecte technique et fonctionnel possédant de solides compétences en UX/UI, Tu dois spécifier une application frontend qui accède à tous les points de terminaison du contrat OpenAPI.
```

```
Pour l'instant nous allons uniquement spécifier la page d'accueil qui doit présenter la liste des clients sous forme moderne (de préférence en mode carte) avec les informations suivantes : Nom, Prénom, Code Postal et Ville. Peux-tu également me fournir des maquettes d'écran et un diagramme de séquence Front / Backend ?
```

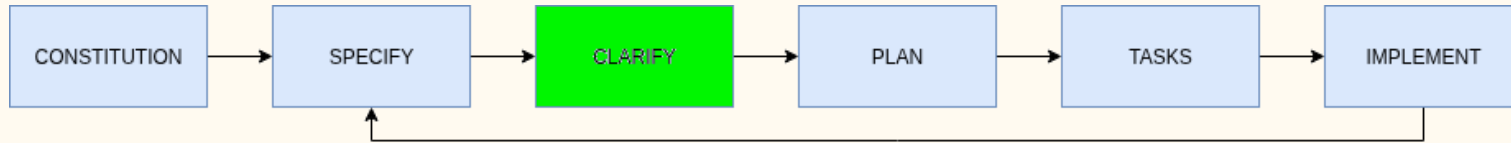
```
Une fois la tâche réalisée, rends moi la main afin que je valide ce que tu auras rédigé.
```



Spécification attendue :

- Consultation liste
- Responsive et UX moderne
- Consultation fiche

Clarification



Exemple de prompt :

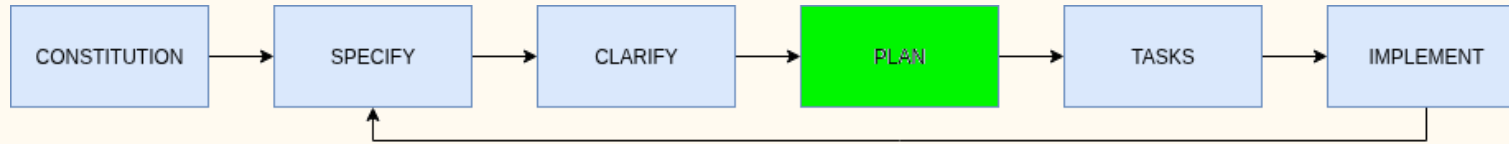
```
/speckit.clarify Clarifies la spécification ci-jointe si nécessaire.
```



Passage en mode interactif

- Pagination ?
- Messages d'erreurs ?

Définir le plan



Exemple de prompt :

```
/speckit.plan Crées un plan pour la spécification ci-jointe. Une fois le plan établi rends moi la main afin que je le valide.
```

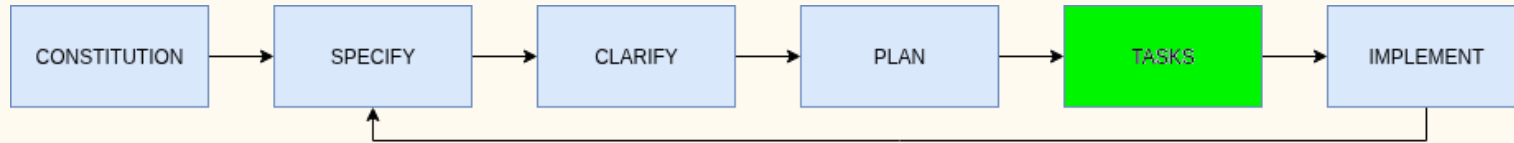


Plan défini

- Phase de recherche
- Phase de design
- Planification



Transformer le plan en tâches



Exemple de prompt :

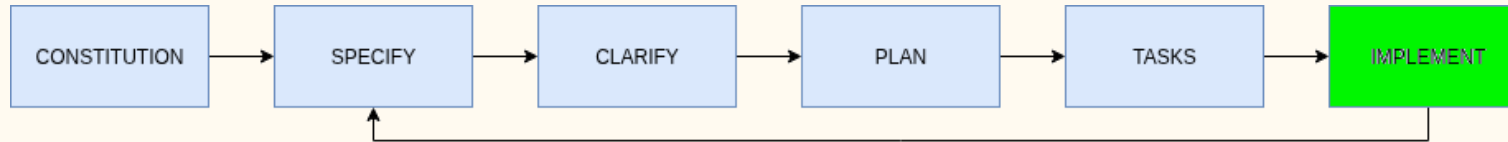
```
/speckit.tasks Transforme le plan ci-joint en tâches. Une fois les tâches établies rends moi la main afin que je les valide.
```

Tâches définies

- 8 phases
- 21 tâches



Démarrer l'implémentation



Exemple de prompts :

```
/speckit.implement Implémente la phase 1, puis rend moi la main afin que je puisse valider, l'idéal serait de pouvoir lancer l'application pour voir le rendu initial.
```

```
/speckit.implement Implémente la phase 2, puis rend moi la main afin que je puisse valider, l'idéal serait de pouvoir lancer l'application pour voir le rendu.
```

```
/speckit.implement Implémente la phase 3, puis rend moi la main afin que je puisse valider, l'idéal serait de pouvoir lancer l'application pour voir le rendu.
```

```
/speckit.implement Implémente le reste des phases, puis rend moi la main afin que je puisse valider, l'idéal serait de pouvoir lancer l'application pour voir le rendu.
```

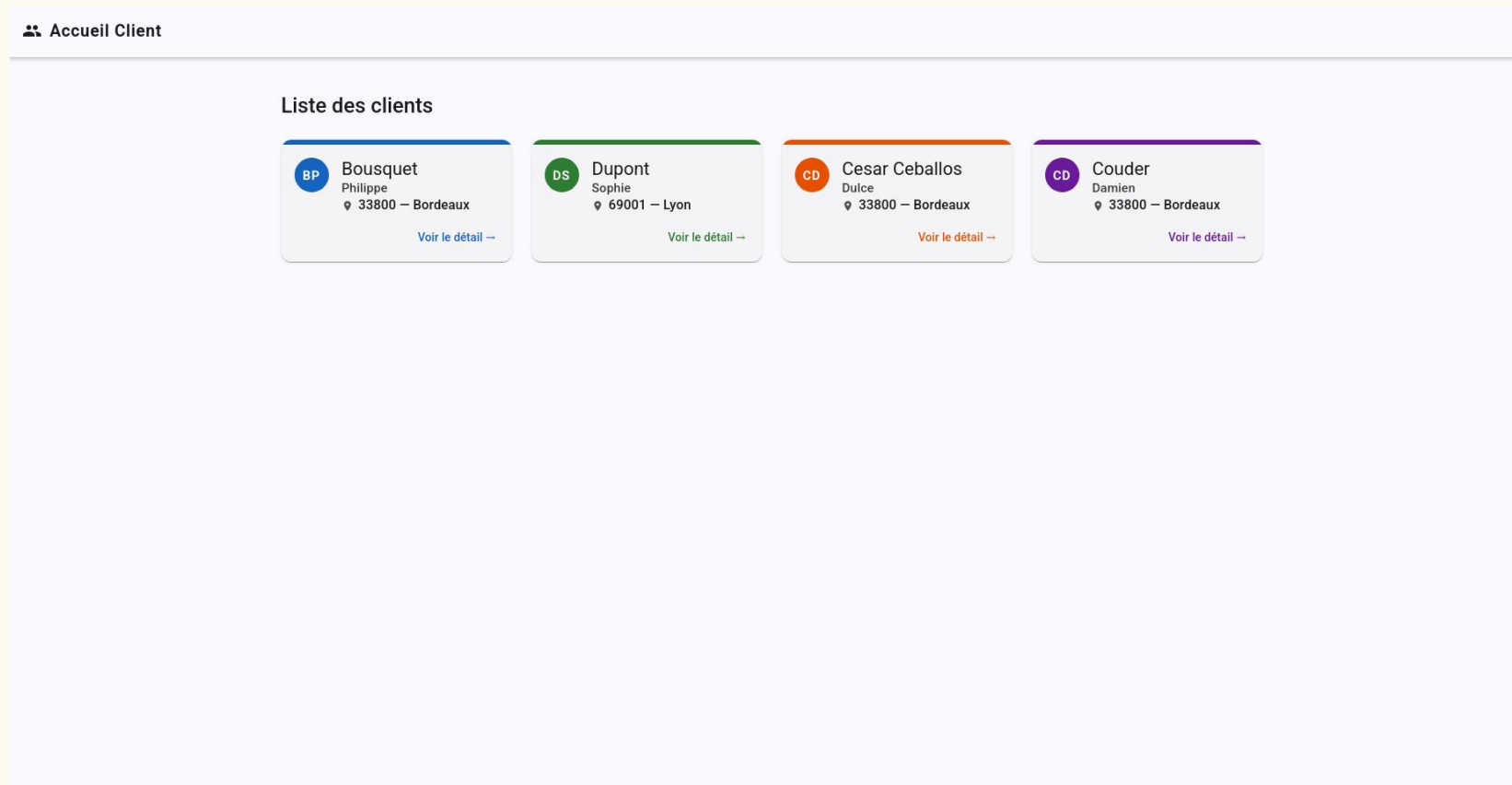
Passage en cycle itératif

- Phase 1 -> Setup
- Phase 2 -> Fondations
- Phase 3 -> User Story 1
- ...

Exécuter l'application

<http://localhost:4200/>

ng serve





Connaissance Client

Reprise d'une application existante (Backend API)

Initialisation de Spec-Kit

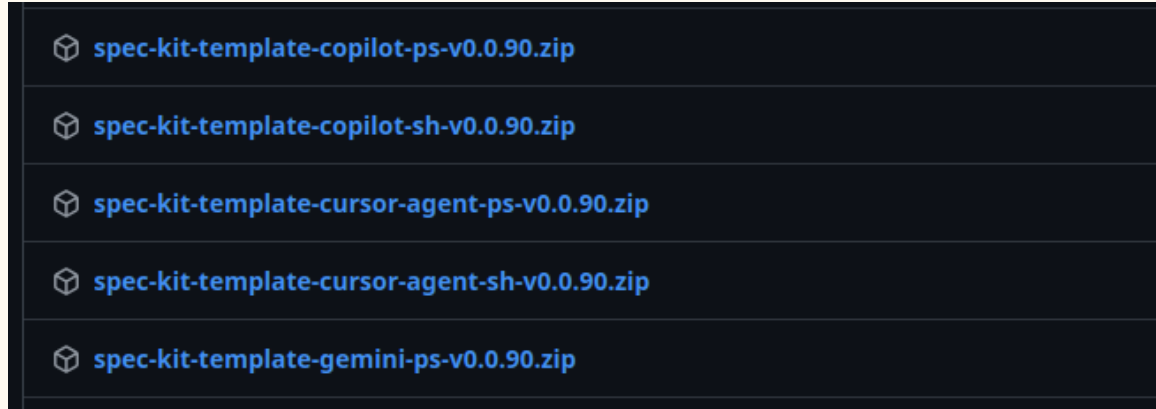
Via la CLI

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git
```

```
cd connaissance-client && specify init --here
```

Ou installation Manuelle

Depuis le repo : <https://github.com/github/spec-kit/tags>

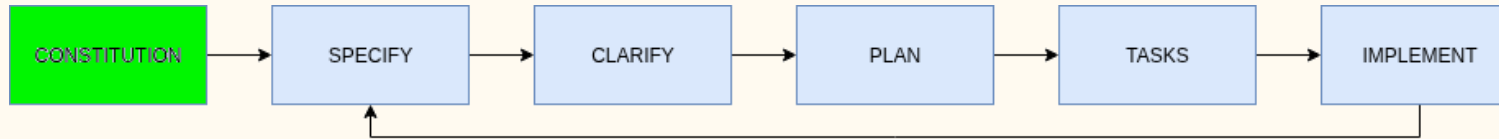


Décompresser l'archive dans votre projet.



```
▼ CONNAISSANCE-CLIENT
  ▼ .github
    ▼ agents
      ≡ speckit.analyze.agent.md
      ≡ speckit.checklist.agent.md
      ≡ speckit.clarify.agent.md
      ≡ speckit.constitution.agent.md
      ≡ speckit.implement.agent.md
      ≡ speckit.plan.agent.md
      ≡ speckit.specify.agent.md
      ≡ speckit.tasks.agent.md
      ≡ speckit.taskstoissues.agent.md
    > prompts
    > .mvn/wrapper
  ▼ .specify
    > scripts
    > templates
    > .vscode
```

Constitution



Exemple de prompt :

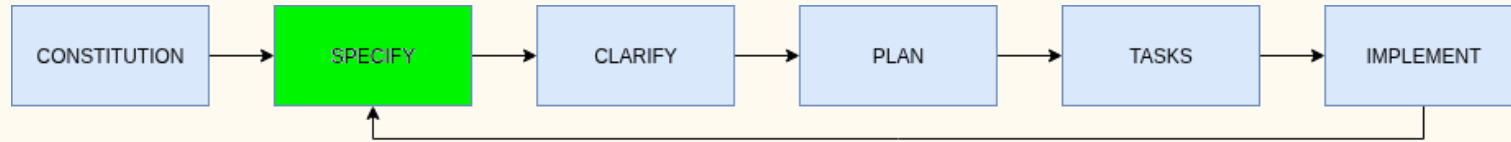
```
/speckit.constitution En tant qu'architecte technique spécialisé en Java et Spring-Boot, effectue une analyse complète du projet afin d'en déterminer les principes d'architecture et les pratiques de développements afin d'alimenter le constitution.md. Une fois la constitution rédigée, rends-moi la main afin que je procède à sa validation avant de passer à l'étape suivante.
```



Principes attendus :

- Domain Driven Design
- Architecture Hexagonale
- API-First Contract
- TDD
- Java 21, Spring Boot 4

Rétro-Spécification



Exemple de prompt :

```
/speckit.specify En tant qu'architecte technique et fonctionnel  
spécialisé en Java et Spring-Boot, effectue une analyse complète du code  
de ce projet afin d'en faire une rétro spécification, appuies toi en  
premier lieu sur les contrats OpenApi, AsyncApi et Les Modèles de  
données. Effectue également un schéma d'architecture pour présenter les  
interactions entre les différents composants. Rends-moi la main, afin  
que je procède à sa validation avant de passer à l'étape suivante.
```



Spécification attendue :

- Entité clé: Client
- Endpoints principaux:
 - GET (liste)
 - POST (creation)
 - GET {id}
 - DELETE {id}
 - PUT {id}/adresse
 - PUT {id}/situation
- Validation adresse via IGN.
- PUT adresse -> Kafka
- Persistance: MongoDB

Demo / VS Code

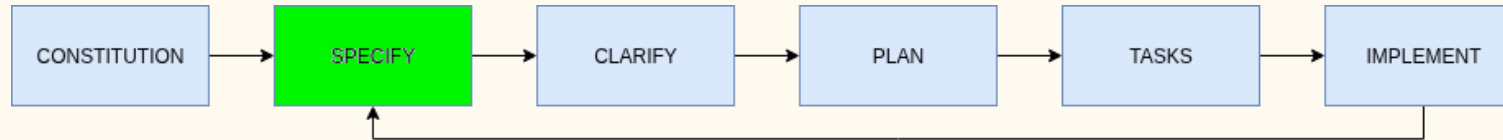


Connaissance Client

Ajout d'une nouvelle fonctionnalité



Nouvelle fonctionnalité



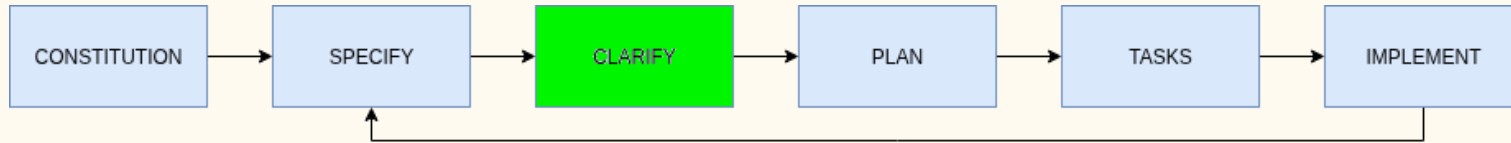
Exemple de prompt :

```
/speckit.specify Nous allons développer une nouvelle version (2.3.0-SNAPSHOT) avec une nouvelle fonctionnalité. Peux tu proposer en respectant l'ensemble des principes de constitution et de retro-spec précédemment définis un nouvel endpoint de modification partielle du client (nom et/ou prénom) en utilisant le verbe http PATCH ? Une fois la spécification rédigée, rends moi la main afin que je la valide.
```

Spécification attendue :

- Version 2.3.0-SNAPSHOT
- Endpoint PATCH {id}
- 3 US stories
- Contract First OpenApi

Clarifier la spécification



Exemple de prompt :

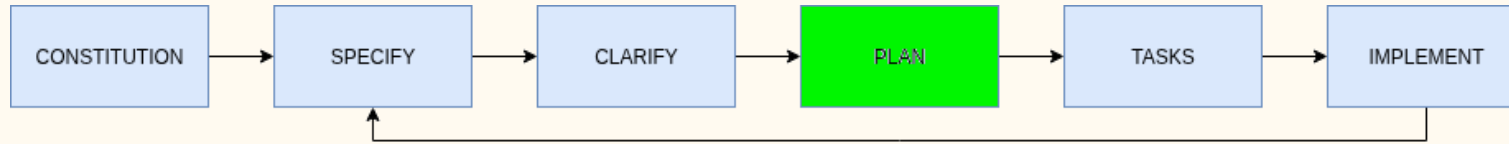
```
/speckit.clarify Clarifies la spécification ci jointe si nécessaire  
(nouvel endpoint de modification client avec le verbe http PATCH).
```



Passage en mode interactif

- Concurrence ?
- Messages d'erreurs ?

Définir le plan



Exemple de prompt :

```
/speckit.plan Crées un plan pour la spécification ci-jointe. Une fois le plan établi rends moi la main afin que je le valide.
```

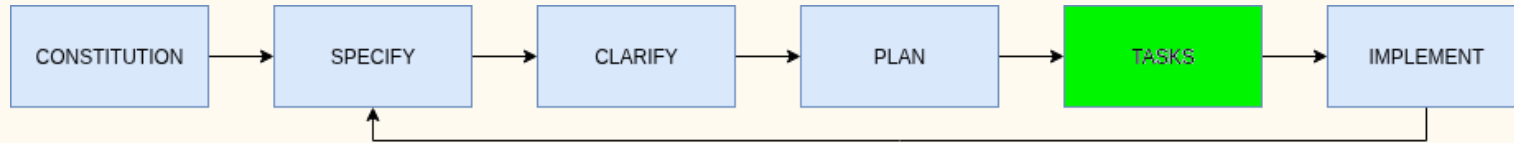


Plan défini

- Finaliser l'OpenAPI
- Écrire tests contractuels
- Implémenter/adapter
- Définir l'approche atomicité/compensation
- Tests d'intégration
- CI
- Documentation



Transformer le plan en tâches



Exemple de prompt :

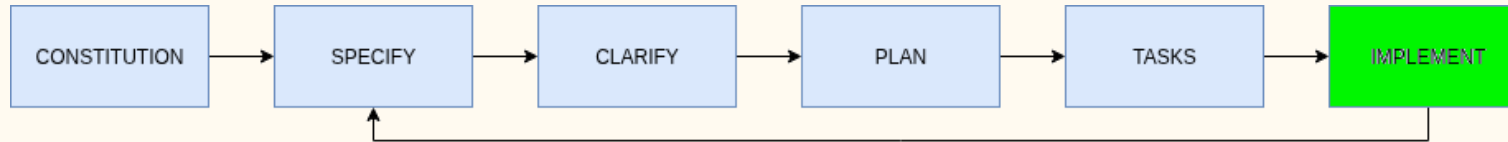
```
/speckit.tasks Transforme le plan ci-joint en tâches. Une fois les tâches établies rends moi la main afin que je les valide.
```

Tâches définies

- 7 phases
- 3 US stories
- 26 tâches



Démarrer l'implémentation



Exemple de prompt :

```
/speckit.implement Démarres la mise en oeuvre par phases. Après chaque phase propose moi une étape de validation. Une fois validée, effectues un commit avant de passer à la phase suivante.
```

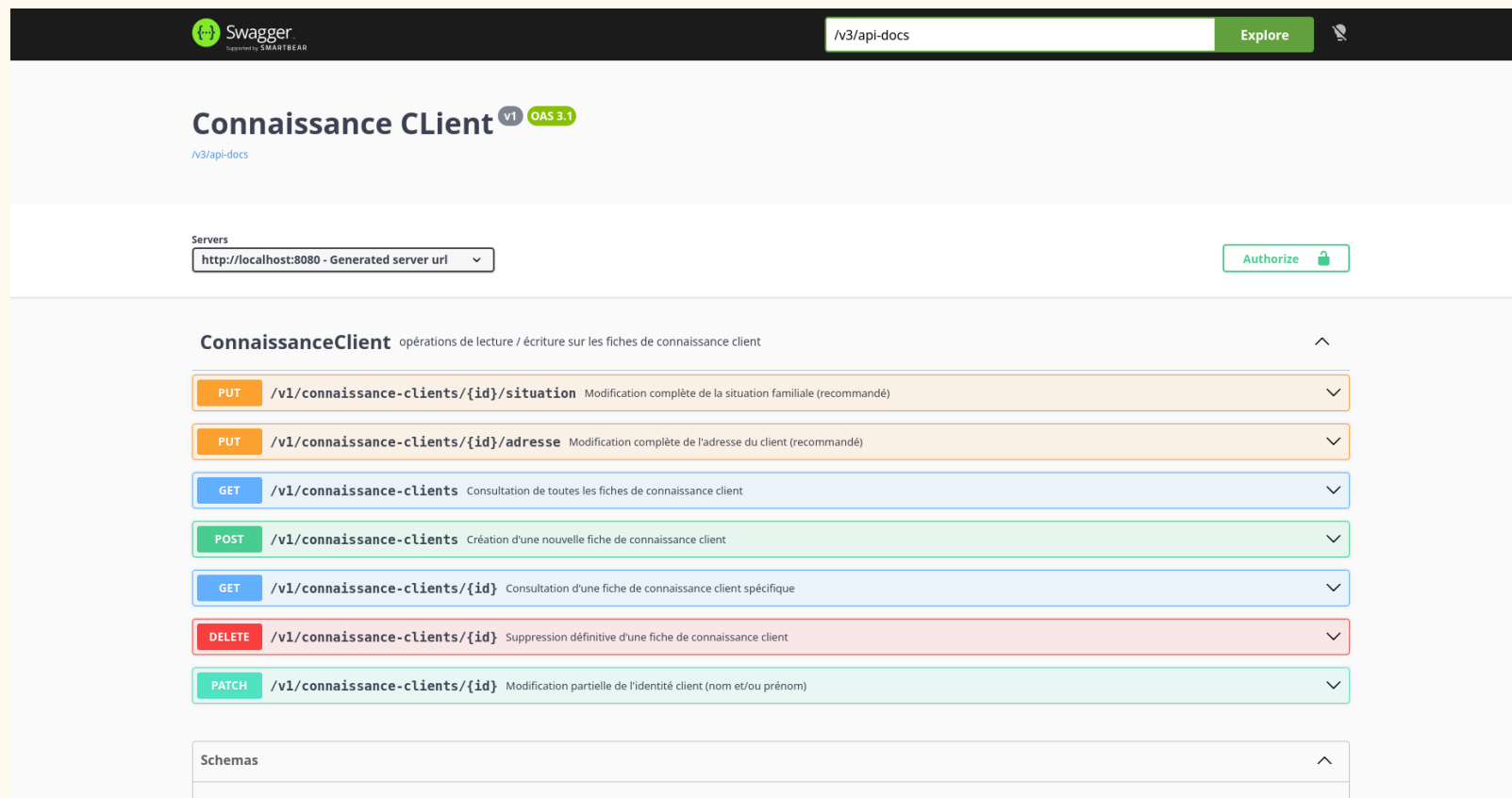
Passage en cycle itératif

- Design first OpenApi
- Implémentation métier
-

Exécuter l'application

<http://localhost:8080/swagger-ui/index.html>

```
java -jar ./connaissance-client-app/target/connaissance-client-app-2.3.0-SNAPSHOT.jar
```





D'autres fonctionnalités ?

Laissez libre court à votre imagination

Exemples Backend

- Ajouter un numéro de téléphone
- Ajouter un email
- Mettre en place le contrôle JWT
- ...

Exemples Frontend

- Mettre en place les autres écrans
- Mettre en place un Logo
- Mettre en place l'authentification
- ...

Conclusion

À retenir

À retenir

- Spec-Kit, un workflow simple et structuré pour le Specification-Driven Development
- Essayez-le, testez-le, pratiquez-le : c'est en l'utilisant qu'il prend tout son sens
- Commencez petit : le [String Calculator Kata](#) est un excellent point d'entrée
- Attention aux limites : les technos sorties après la date d'entraînement du LLM
- Le **rétro-engineering** fonctionne très bien sur des applications bien architecturées (Hexagonal, DDD, etc.)
- **Spécifiez en langage maternel** : vous serez souvent plus précis
- Pour mieux spécifier, **pensez design-first** : **OpenAPI, AsyncAPI, Data Models** avant le code

sql

DEVOXX™ France

Thank You



Elevate. Digitally